INFORMATION SECURITY · PORTFOLIO

Seungchan Song



I test systems the way attackers do, and explain the risk with law and data.

I built RAG-DIAG, a tool that diagnoses personal-data leakage from AI chatbots, and filed it as a patent. I have practised system hacking and web penetration testing, and I document every finding with how to reproduce and fix it.

chanchan3314@naver.com 010-7406-3317 github.com/seungchan-song 2001.01.04

Education	Sejong University, Information Security (Mar 2021 – Feb 2027, expected) · GPA 4.10 / 4.5 Sangsan High School (Mar 2017 – Feb 2020)
Credentials	Engineer Information Security (Aug 2026) · TOEIC 850 (Feb 2026) · OPIc IH (Sep 2026)
Abroad	Shenzhen Funful Bilingual School · Shenzhen, China (Apr 2006 – Jul 2009) Green Oasis School · Shenzhen, China (Nov 2009 – Oct 2011) UBC Blockchain Summer Institute · Vancouver, Canada (Aug 2026)
Languages	English (proficient) · Chinese (intermediate)
Skills	Languages: Python · C · Linux AI security: RAG · Haystack · FAISS Security testing: Burp Suite · Nmap · Nikto · GDB · pwntools
Military	Republic of Korea Air Force (Apr 2023 – Jan 2025)

PATENT
Patent filed
RAG leakage diagnosis

AWARD
3rd prize
Capstone competition

PROGRAM
Elite Top 3
System-hacking program

Projects

01	**RAG-DIAG**	AI chatbot data-leak diagnosis tool · patent filed	p.2
02	**Training-site pen test**	Black-box web pen test · five checks	p.3
03	**Health Access Registry**	UBC blockchain hackathon · medical-record access	p.4
04	**Voice-phishing Radar**	Automated fraud-trend analysis	p.5
05	**System-hacking program**	12-week course · Elite Top 3	p.5

PROJECT 01 · AI SECURITY

RAG-DIAG

github.com/seungchan-song/RAG-DIAG ↗

A CLI tool that diagnoses personal-data leakage from AI chatbots (RAG)

When **2026.03 – 2026.08** Team **4 people · Sejong capstone** Role **Lead · detection, judging, risk scoring**

Problem & goal

Chatbots that answer from retrieved documents can leak the personal data inside them. We built a tool that sends attack questions, finds personal data in the answers and measures the risk.

What I contributed

01 Team lead · schedule and meetings

Three-month WBS for roles and deadlines; weekly meetings for progress, a separate slot for decisions.

02 Four-stage Korean PII detection

English-centric tools miss Korean identifiers. Regex → checksum → NER → sLLM cross-check, with the attack-success rule built on top.

03 One risk scale for every scenario

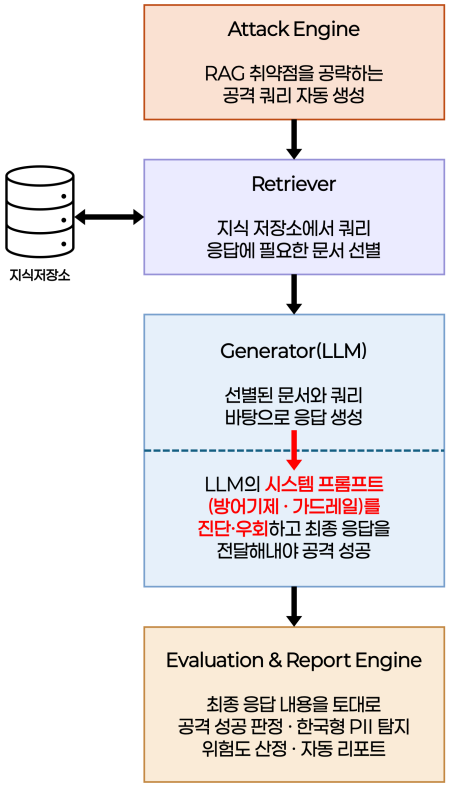
Severity graded under Korea's privacy law plus success rate gives one score, so four different attacks can be compared and prioritised.

04 Proposed a control group

Ordinary questions served as a control, so only the extra leakage caused by attacks was counted.

05 Documentation and patent

Led the design docs; wrote the invention description and prior-art comparison and took the patent to filing (co-inventor).



Architecture

Results

PATENT

System and Method for Diagnosing Information Leakage Vulnerabilities in RAG Environment

No. 10-2026-0164171 · filed 31 Aug 2026

Applicant: Sejong Univ. · examination requested, under review

AWARD

3rd prize

Capstone competition · Jun 2026

OPEN SOURCE

Contest entry

2026 open-source contest

PROJECT 02 · WEB PENETRATION TEST

Web penetration test of a training shop

Black-box testing of a mock shopping site built for training, written up as a full report

When	Team	Role
28–29 Oct 2026	7 people · SK shieldus Rookies 35	Five checks · fixes

Problem & goal

The team split 30 checks in 17 categories, based on the OWASP Top 10 and KISA's key vulnerabilities, and produced one report from reconnaissance through exploitation to fixes. I owned five checks covering server configuration, information exposure and compliance with Korea's Personal Information Protection Act.

What I contributed

Recon → Analysis → Exploitation → Impact → Fixes

Check	What I found	Recommended fix
Plaintext transfer	Profile updates sent the ID and password in plaintext; the admin page showed buyers' IP addresses	Force HTTPS; Secure, HttpOnly and SameSite cookies
Unneeded pages	DIRB and Nikto found phpinfo.php open without login, exposing PHP, OS and kernel versions	Delete diagnostic pages; require login and IP limits for any that must stay
Test and demo pages	Checked backup extensions, robots.txt and listable paths; flagged phpinfo.php for removal	Remove test pages; disable directory listing
Server configuration	Headers leaked Apache, PHP and OpenSSL versions and WebDAV; security headers missing; TRACE allowed	Hide versions (ServerTokens, expose_php), disable TRACE, add security headers
Privacy-law compliance	Seven gaps, such as collecting resident numbers, plaintext transfer, unsalted passwords and bundled consent, each matched to the relevant article of the Act	Stop collecting resident numbers; encrypt in transit and at rest; mask; separate consent; update the policy

Tools: Burp Suite · Nikto · DIRB · Nmap

Results

TEAM
172
Vulnerabilities found by the team (53 high)

MINE
5 checks owned
Plaintext, unneeded pages, test pages, server config, privacy law

COMPLIANCE
7 gaps
Matched to articles of the privacy act

PROJECT 03 · UBC BLOCKCHAIN HACKATHON

Health Access Registry

github.com/seungchan-song/blockathon ↗

Patients control who may open their medical record, and every access attempt is kept tamper-proof

When **Aug 2026 · UBC, Canada**

Team **4 · Korea · Switzerland · China**

Role **Team lead · service design · deployment**

Problem & goal

Hospitals keep the access logs for medical records. To learn who opened their file, patients must ask the hospital itself, and edits or deletions are hard to notice. We set out to record permissions and access attempts on a tamper-proof public ledger, so patients can check for themselves.

What I contributed

01 Flagged the first design

The first plan put encrypted records on-chain. I raised that data on a chain can never be removed, so even encrypted records could not honour a patient's request to delete them.

02 Drew the on-chain line

Rule: anything that may need erasing stays off-chain. Only allowed addresses and access attempts go on-chain; no names, diagnoses or history.

03 No admin; refusals logged

An admin could change permissions without the patient knowing, so only the patient's wallet can. Refused attempts are logged too, turning an access list into an audit trail.

04 AI-assisted build, hands-on checks

The code was written with AI tools, but I first wrote the design rules as a guideline: what goes on-chain and who holds authority. I then spent time checking grant, revoke and refusal logging scenario by scenario.

Limits & next steps

Doctor identity

We cannot prove a wallet belongs to a real doctor; verifiable credentials can add that without changing the contract.

Metadata exposure

Names stay hidden but addresses and times are public; we proposed zero-knowledge proofs to verify access without revealing who looked.

Results

DEPLOYMENT

Live on testnet

Deployed to Sepolia; live demo of denied → granted → on-chain log

PROJECT 04 · DATA AUTOMATION

Voice-phishing Radar

github.com/seungchan-song/voicephishing-radar ↗

When **2026.09 – 2026.10** Team **6 · SK shieldus Rookies** Role **Lead · design · integration**

Problem & goal

Police statistics show past voice-phishing damage by age, region and year; news shows today's tactics, which helps prevent the next losses. We collected and analysed both automatically every week.

What I contributed

01 One results file
All processed statistics and news go into one result.json that the web, mail and Notion read, so each feature can be built and fixed on its own.

02 End-to-end automation
My shared code and run-all script run everything from crawling to dashboard, mail and Notion updates every Monday.

03 Three outputs by role
Dashboard for status, alert mail for new tactics, Notion for the weekly record.

What I learned

Designing for automation
One scheduled pipeline from crawling to alerts taught me to design for automation from the start.

Recording why
The README records why there is one results file (easier development) and three outputs (separate roles), so anyone can extend it.

PROJECT 05 · SYSTEM HACKING

P4C cohort 13, system hacking (RED course)

When **2025.06 – 2025.08** Format **Individual · 12 weeks** Completion **31.8%**

Learning goal

To understand how attackers manipulate program memory, and to be able to analyse and exploit binaries with protections enabled.

What I did

01 Memory and execution
Stack and heap layout, assembly and executable formats, traced step by step in GDB (pwndbg).

02 Exploits and bug hunting
Reproduced overflows and ROP on wargames to learn protection bypasses; found bugs with fuzzers and code review.

03 Written notes
Wrote up every week's theory and assignments in Notion, plus my own pwnable reference.

Results & lessons

RESULT

Elite Top 3
Top 3 of all finishers

LESSON

From attack to defence
Having manipulated memory myself, I look first for missing bounds checks and input validation.

TIMELINE

Timeline

Awards · Projects · Credentials · Ongoing & upcoming · Education & other

2006	2006.04 – 2009.07	Shenzhen Funful Bilingual School · Shenzhen, China
2009	2009.11 – 2011.10	Green Oasis School · Shenzhen, China
2017	2017.03 – 2020.02	Sangsan High School
2021	2021.03	Entered Sejong University, Information Security
2023	2023.04 – 2025.01	Republic of Korea Air Force
2025	**2025.06 – 2025.08**	**P4C cohort 13, system hacking · Elite Top 3**
2026	2026.02	TOEIC 850
	2026.03 – 2026.08	**RAG-DIAG · capstone competition, 3rd prize**
	2026.08	**UBC Blockchain Summer Institute, Canada · hackathon**
	2026.08	Engineer Information Security
	2026.08	**Patent filed · RAG leakage diagnosis**
	2026.09 – now	**SK shieldus Rookies 35, cloud-based converged security course**
	2026.09	OPIc IH
	2026.09 – 2026.10	**Voice-phishing Radar**
	2026.10	**Web penetration test of a training shop**
2027	**2027.02**	**Graduating from Sejong University**

Thank you for reading.

chanchan3314@naver.com 010-7406-3317 github.com/seungchan-song